

P4211R0

Adaptors For Closed Ranges

Published Proposal, 2026-05-05

This version:

<https://wg21.link/P4211R0>

Author:

[Yihe Li](#)

Audience:

SG9

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

C++29

Source:

github.com/Mick235711/wg21-papers/blob/main/P4211/P4211R0.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

1 Revision History

1.1 R0 (2026-05 pre-Brno mailing)

2 Background

3 Design

3.1 What Is Being Proposed?

3.2 Prior Art

3.3 Naming

3.4 Why Not A Breaking Change?

3.5 Range Properties

3.5.1 Category

3.5.2 Common

3.5.3 Sized

3.5.4 Const-Iterable

3.5.5 Borrowed

3.6 Feature Test Macro

3.7 Freestanding

4 Implementation Experience

5 Wording

5.1 17.3.2 Header `<version>` synopsis [version.syn]

5.2 24.2 Header `<iterator>` synopsis [iterator.synopsis]

5.3 24.5.❖ Lazy counted iterators [iterators.lazy.counted]

5.3.1 24.5.❖.1 Class template `lazy_counted_iterator` [lazy.counted.iterator]

5.3.2 24.5.❖.2 Constructors and conversions [lazy.counted.iter.const]

5.3.3 24.5.❖.3 Accessors [lazy.counted.iter.access]

5.3.4 24.5.❖.4 Element access [lazy.counted.iter.elem]

5.3.5 24.5.❖.5 Navigation [lazy.counted.iter.nav]

5.3.6 24.5.❖.6 Comparisons [lazy.counted.iter.cmp]

5.3.7 24.5.❖.7 Customizations [lazy.counted.iter.cust]

5.4 25.2 Header `<ranges>` synopsis [ranges.syn]

5.5 25.7.❖ Lazy take view [range.lazy.take]

5.5.1 25.7.❖.1 Overview [range.lazy.take.overview]

5.5.2 25.7.❖.2 Class template `lazy_take_view` [range.lazy.take.view]

5.5.3 25.7.❖.3 Class template `take_view::sentinel` [range.lazy.take.sentinel]

5.6 25.7.❖ Lazy counted view [range.lazy.counted]

5.7 25.7.❖ As closed view [range.as.closed]

5.7.1 25.7.❖.1 Class template `as_closed_view` [range.as.closed.view]

5.7.2 25.7.❖.2 Class `as_closed_view::iterator` [range.as.closed.iterator]

References

Normative References

Informative References

This proposal introduces a family of adaptors that convert closed ranges into half-open ranges, as expected by most other standard library facilities in C++, thus providing direct support for a range model that has been fundamentally incompatible with the C++ iterator model until now.

§ 1. Revision History

§ 1.1. R0 (2026-05 pre-Brno mailing)

- Initial revision.

§ 2. Background

Ever since C++98, ranges in C++ are modeled by half-open intervals `[a, b)`, and most other facilities in the standard library are built with this assumption in mind. All standard algorithms and containers accept an iterator pair that contains a *past-the-end* iterator, not the iterator pointing to the last element. Though this may seem unintuitive at first to many beginners, the adoption of half-open ranges has led to more intuitive behavior in many cases, such as avoiding +1/-1 adjustments when slicing.

However, despite such regulations, closed ranges are still occasionally a useful abstraction to have. For instance, `views::iota(INT_MAX - 20, INT_MAX)` will only give you numbers up to `INT_MAX - 1`, since `iota`'s arguments also follow the half-open range abstraction. In this case, there is no way to obtain an `iota_view` that includes the largest value representable by its value type, because signed integer overflow is undefined behavior.

A more common example can be found in [\[P2406R5\]](#):

```
auto iss = std::istringstream("0 1 2");
for (auto i : views::istream<int>(iss) | views::take(1))
    std::cout << i << '\n';
auto i = 0;
iss >> i;
assert(i == 1); // FAILS, i == 2
```

At first glance, this code looks like it should work, as we are taking one element from the `istream_view` and thus should consume one integer from the string. However, as `take_view` uses `counted_iterator` internally, which still advances the underlying iterator even when `count == 0`, an additional integer is consumed and subsequently lost.

This unintuitive behavior is a direct result of lacking an abstraction for closed ranges. Essentially, we are trying to create a range of 1 element, but half-open ranges need 2 iterator values to represent such ranges, thus consuming one more iterator value from the `istream_view`. Similarly, the other example presented in [\[P2406R5\]](#) can also be resolved by a proper abstraction for closed ranges:

```
for (auto i : views::iota(0)
    | views::filter([](auto i) { return i < 11; })
    | views::take(11))
    std::cout << i << '\n'; // Infinite loop!
```

Conceptually, we are also taking 11 elements from a range of 11 elements; however, to present the range as a half-open one, we actually need 12 iterator values. Computing the 12th iterator value ultimately results in an infinite loop, as there is no further value available after filtering.

In light of these examples, this proposal introduces a family of adaptors for closed ranges to convert them into half-open ones. Using the facilities proposed by this proposal, the above examples can be rewritten as:

```
auto iss = std::istringstream("0 1 2");
for (auto i : views::istream<int>(iss) | views::lazy_take(1))
    std::cout << i << '\n';
auto i = 0;
iss >> i;
assert(i == 1); // OK
```

```
for (auto i : views::iota(0)
    | views::filter([](auto i) { return i < 11; })
    | views::lazy_take(11))
    std::cout << i << '\n'; // OK

for (auto i : views::iota(0)
    | views::filter([](auto i) { return i < 11; })
    | views::take(10) | views::as_closed)
    std::cout << i << '\n'; // Also OK
```

§ 3. Design

§ 3.1. What Is Being Proposed?

This proposal is essentially a continuation of [\[P2406R5\]](#) with a broader scope. We propose the same facilities that are proposed in [\[P2406R5\]](#):

- A `std::lazy_counted_iterator` (and the corresponding `views::lazy_counted` factory) that avoids incrementing the underlying iterator when `count == 0`.
- A `views::lazy_take` adaptor that uses `lazy_counted_iterator` as its underlying iterator in non-random access cases.

Additionally, we also proposes a general adaptor, `views::as_closed`, for all kinds of closed ranges (basically a range such that `*iter` is still valid even when `iter == end`). Such an adaptor is useful when a count is not readily available.

Finally, we also proposes a convenient alias `views::closed_iota` that is equivalent to `views::as_closed` plus `views::iota`, as a convenient way to create closed integer interval ranges.

§ 3.2. Prior Art

Looping over a closed range is actually pretty hard to get right, as evidenced by the multiple unintuitive methods suggested [on StackOverflow](#):

```
// Half-open ranges are easy to iterate
for (auto it = begin; it != end; ++it) { ... }

// What if the range is closed?
// Only valid with random access iterator, and become infinite/UB when end
for (auto it = begin; it <= end; ++it) { ... }

// end + 1 need to actually be valid
for (auto it = begin; it != std::next(end); ++it) { ... }

// Not valid for input iterators, and still UB when end is the max value
auto it = begin; do { ... } while (it++ != end);

// You must resort to one of
auto it = begin; do { ... } while (it != end && (++it, true));
```

```

for (auto it = begin; ; ++it) { ...; if (it == end) break; }

// Or repeat the body
auto it = begin;
for (; it != end; ++it) body(*it);
body(*it);

```

Such unintuitiveness is effectively what leads to range-v3's `views::closed_iota` adaptor, which provides a closed range version of `views::iota`.

Obviously, such abstractions must incur some kind of overhead, since we need to pull out a past-the-end iterator value from thin air somehow. `closed_iota_view`'s iterator represents this new value via a `bool` member:

```

struct closed_iota_view<W, Bound>::iterator
{
    W current;
    Bound bound;
    bool past_the_end;

    iterator& operator++()
    {
        if (current == bound) past_the_end = true;
        else ++current;
        return *this;
    }
};

```

Not only is there an extra `bool` value, but each iterator also needs to store the bound value too (which is not needed in normal `iota_view`'s iterator). Furthermore, each increment needs to compare the current value to the bound value to determine if an increment is needed. However, these overheads are required to adapt an abstraction that is essentially foreign to the C++ iterator model.

Given that we need to pay certain overheads already, it seems unwise to limit such a closed adaptor to `views::iota` only; this line of thought naturally leads to a general closed-to-half-open range adaptor:

```

struct as_closed_view<I, S>::iterator
{
    I current;
    S last;

```

```

    bool past_the_end;

    iterator& operator++()
    {
        if (current == last) past_the_end = true;
        else ++current;
        return *this;
    }
};

```

The iterator itself is nearly identical, with the only difference being that `operator*` actually dereference the iterator. With such a general adaptor, we can then construct arbitrary closed ranges easily:

```

std::vector vec{1, 2, 3, 4, 5};
views::as_closed(vec.begin() + 2, vec.begin() + 4); // [3, 4, 5]

```

Meanwhile, `closed_iota` can now be respecified as a simple combination of `as_closed_view` and `iota_view`, with the exact same memory layout and footprint, thereby requiring one less specification for adaptors. It is for this reason that this proposal argues for a general adaptor instead of proposing to standardize range-v3's `views::closed_iota`.

§ 3.3. Naming

The exact-N version of `counted_iterator` and `take_view` are named with `lazy_` prefix, similar to [P2406R5]'s choice. The reasoning for this choice is that `lazy_` prefix has a preexisting meaning in the standard library to be maximally lazy; for instance, `lazy_split_view` (introduced by [P2210R2]) will never touch any element until you increment to the element itself (i.e., it will not compute the next pattern's position beforehand, unlike `split_view`), which gives it the ability to support input ranges. Similarly, `lazy_counted_iterator` will never increment the underlying iterator past the given N positions.

The newly proposed general closed to half-open range adaptor is named `views::as_closed` (and `ranges::as_closed_view`). Specifically:

- `views::as_closed(begin, end)` will adapt the given iterator-sentinel pair into a normal half-open range by inventing an extra sentinel position.
- `rng | views::as_closed` or `views::as_closed(rng)` will adapt the given range into a normal half-open range, as if by `views::as_closed(ranges::begin(rng), ranges::end(rng))`.

Such naming follows the precedent set by `views::as_input` ([P3828R1]), where `as_` prefix is used to signal that the underlying range is seen *as* something different.

Finally, `views::closed_iota` is chosen as the name for the convenient alias for closed value ranges, following the precedent of the naming in `range-v3`. The author considered several shorter names, including `views::upto`, `views::range` (like Python `range()`), and `views::ints`, but it seems that most of these names are ambiguous in terms of whether the resulting range is inclusive or exclusive. (`views::range` also has potential to be confused with the `range` concept, so perhaps NumPy-like `views::arange` is more appropriate; however, both `range()` and `arange()` are exclusive in Python.)

It is worth noting that one of the opposition comments expressed to `views::upto` proposed by [P3060R1] was that `upto` is meant to be inclusive in English; so the author is also happy to change to these kinds of names if requested by SG9/LEWG.

§ 3.4. Why Not A Breaking Change?

During the review of [P2406R5], LEWG expressed interest in treating the proposal as a breaking change: change the existing `counted_iterator` and `take_view` to have lazy semantics instead, as many people suggests that `views::take(N)` should only consume `N` iterator values, regardless of the underlying abstraction. An NB comment [US46-107] was submitted to do this change for C++23; however, the comment was ultimately rejected together with [P2406R5] despite having consensus to do a breaking change in LEWG due to lack of design time.

Another proposal, [D2578R0], suggested a narrower breaking change that blocks input iterator's usage with `counted_iterator`, as the result in most cases is wrong (consumes one more iterator value that is not recoverable). However, the proposal also pointed out that any such blank refusal will necessarily also break valid usages of input iterators, such as using `istreambuf_iterator` with `counted_iterator`. Since the former is already lazy (only removing the current element from the stream on next `++`), the extra consumption is perfectly fine. [D2578R0] argues for a new concept, `lazy_weakly_incrementable`, to detect such cases. The proposal was eventually rejected by SG9 due to the inconsistency in treatment.

Standing at two standard cycles later, C++20 Ranges and `views::take` have already been in deployment for more than 6 years, and it seems no longer wise to pursue a breaking change in this regard. Furthermore, [P2799R0] pointed out that the underlying desire is some kind of adaptation from closed ranges into half-open ranges, and "fixing" the existing `counted_iterator` in most cases will only add unwanted performance overhead, which is not the correct way to support closed ranges.

In light of these discussions, this proposal does not pursue a breaking change to any existing facility. Instead, new families of lazy `counted_iterator`, `take_view`, and a new general adaptor for closed ranges are proposed, returning the choice to the user and highlighting the potential overhead by requiring explicit adaptation.

§ 3.5. Range Properties

The proposed `views::lazy_counted` and `views::closed_iota` views are range factories, i.e., cannot be piped into. This is consistent with the existing `views::counted` and `views::iota` as the first argument is an iterator. (Note that `views::closed_iota` always accepts two arguments, as the infinite variant makes no sense as a closed range.)

The proposed `views::as_closed`, on the other hand, is a range adaptor, i.e., can be piped into. It is treated as a *range adaptor closure object* so that the two-argument version is not picked up in pipelines.

§ 3.5.1. Category

SG9 discussion on [P2406R5] had consensus that the proposed `lazy_counted_iterator` should be capped to an forward iterator. The reason for the lack of bidirectional traversal is the requirement for supporting construction with 0 as count; such construction puts the iterator into an inconsistent state, as the underlying iterator is expected to be "one step back", as explained in section 6.1 of [P2406R5]. As a result, `base()` is also removed from the iterator's API to prevent the user from observing the inconsistent state. Similarly, for non-random-access or non-sized inputs, `views::lazy_take` also degrades to a forward range at most.

It is worth noting that the existing optimizations for `views::counted` and `views::take` (i.e., preserving the input range type when the input range is `empty_view` or similar) still hold, so random access iterator inputs still preserve their category.

Such limitations in general do not apply to `as_closed_view`, as it does not support construction with `past_the_end == true`; the only way to construct an iterator with that state is through `end()` (with common range inputs), where we can guarantee the position of the underlying iterator. Therefore, we propose that `views::as_closed` retain the underlying iterator's category (except contiguous).

Finally, as [P2799R0] pointed out, postfix `operator++` can return a proxy for input or output iterators, so we cannot support them in closed range adaptors. For this reason, both `lazy_counted_iterator` and `as_closed_view` only declare `void operator++(int)` when

the underlying iterator is only input, and does not support output iterators at all. This decision makes them invalid C++17 input iterators, so `iterator_category` is not defined when the underlying iterator is only input.

§ 3.5.2. Common

For `views::lazy_counted`, if and only if the given iterator is random access, due to the usage of `ranges::subrange` in wrapping.

For `views::lazy_take`, if and only if the underlying range is both random access and sized. Only in such cases do we reuse the `iter + size` position as the end iterator; in some of the other cases, `lazy_take_view` will need to add its own sentinel to allow early termination. This design is consistent with `take_view`.

For `views::as_closed`, if and only if the underlying range is common. In such cases we can just pass `end(underlying)` as the current iterator value and manually set `past_the_end` to `true`.

§ 3.5.3. Sized

For `views::lazy_counted`, always (as we can just return `N`).

For `views::lazy_take`, if and only if the underlying range is sized, as we need to do `min(n, size(underlying))`.

For `views::as_closed`, if and only if the provided sentinel is a sized sentinel. (Returns `end - start + 1` since we are inventing one more value.)

§ 3.5.4. Const-Iterable

For `views::lazy_counted` and `views::as_closed`, always, as we are degrading to iterators.

For `views::lazy_take`, if and only if the underlying range is `const`-iterable.

§ 3.5.5. Borrowed

For `views::lazy_counted` and `views::as_closed`, always, as the iterator stores all the information.

For `views::lazy_take`, if and only if the underlying range is borrowed.

§ 3.6. Feature Test Macro

This proposal added two new feature test macros:

- `__cpp_lib_lazy_counted_iterator` for `lazy_counted_iterator`, `views::lazy_counted` and `views::lazy_take`.
- `__cpp_lib_ranges_as_closed` for both `views::as_closed` and `views::closed_iota`.

Of course, the author is happy to split the FTMs further (such as separate FTMs for each adaptor) if LEWG requests.

§ 3.7. Freestanding

All newly introduced FTMs, iterators, and adaptors should be in freestanding, as most facilities in `<iterator>` and `<ranges>` are already in freestanding after [\[P1642R11\]](#).

§ 4. Implementation Experience

The author implemented this proposal in [beman.closed_view](#) as part of Beman Project. No significant obstacles are observed.

§ 5. Wording

The wording below is based on [\[N5032\]](#), with [\[P3828R1\]](#)'s changes already applied on top.

Most wording for `lazy_counted_iterator` and `lazy_take_view` is copied from [\[P2406R5\]](#) with rebases to the latest working draft.

§ 5.1. 17.3.2 Header `<version>` synopsis [version.syn]

In this clause's synopsis, insert a new macro definition in a place that respects the current alphabetical order of the synopsis, and substituting `20XXYYL` by the date of adoption.

```
#define  cpp_lib lazy_counted_iterator 20XXYYL // freestanding,, also in <
// [...]
#define  cpp_lib ranges as closed      20XXYYL // freestanding,, also in <
```

§ 5.2. 24.2 Header `<iterator>` synopsis [iterator.synopsis]

Modify the synopsis as follows:

```
// [...]
namespace std {
    // [...]
    // [iterators.counted], counted iterators
    template<input_or_output_iterator I> class counted_iterator; // freestanding

    template<input_iterator I>
        requires see below
        struct iterator_traits<counted_iterator<I>>; // freestanding

    // [iterators.lazy.counted], lazy counted iterators
    template<input_iterator I> class lazy_counted_iterator; // freestanding
}
// [...]
```

NOTE: Editor's Note: Add the following subclause to 24.5 Iterator adaptors [predef.iterators], after 24.5.7 Counted iterators [iterators.counted]

§ 5.3. 24.5.1 Lazy counted iterators [iterators.lazy.counted]

§ 5.3.1. 24.5.1.1 Class template `lazy_counted_iterator` [lazy.counted.iterator]

Class template `lazy_counted_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that it keeps track of the distance to the end of its range. It can be used together

with `default_sentinel` in calls to generic algorithms to operate on a range of N elements starting at a given position without needing to know the end position a priori.

[Note 1: The difference between `lazy_counted_iterator` and `counted_iterator` is that the former will not increment the underlying iterator when reaching the end of its range. — *end note*]

Two values `i1` and `i2` of types `lazy_counted_iterator<I1>` and `lazy_counted_iterator<I2>` refer to elements of the same sequence if and only if there exists some integer n such that `next(i1.current, i1.count() + n)` and `next(i2.current, i2.count() + n)` refer to the same (possibly past-the-end) element.

```
namespace std {
    template<input_iterator I>
    class lazy_counted_iterator {
    public:
        using iterator_type = I;
        using value_type = iter_value_t<I>;
        using difference_type = iter_difference_t<I>;
        using iterator_concept = see below;
        using iterator_category = see below; // not always present
        constexpr lazy_counted_iterator() requires default_initializable<I> = (
        constexpr lazy_counted_iterator(I x, iter_difference_t<I> n);
        template<class I2>
            requires convertible_to<const I2&, I>
            constexpr lazy_counted_iterator(const lazy_counted_iterator<I2>& x,

    template<class I2>
        requires assignable_from<I&, const I2&>
        constexpr lazy_counted_iterator& operator=(const lazy_counted_itera

    constexpr iter_difference_t<I> count() const noexcept;
    constexpr decltype(auto) operator*();
    constexpr decltype(auto) operator*() const
        requires dereferenceable<const I>;

    constexpr lazy_counted_iterator& operator++();
    constexpr void operator++(int);
    constexpr lazy_counted_iterator operator++(int)
        requires forward_iterator<I>;

    template<common_with<I> I2>
        friend constexpr iter_difference_t<I2> operator-(
```

```

    const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
friend constexpr iter_difference_t<I> operator-(
    const lazy_counted_iterator& x, default_sentinel_t);
friend constexpr iter_difference_t<I> operator-(
    default_sentinel_t, const lazy_counted_iterator& y);

template<common_with<I> I2>
    friend constexpr bool operator==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
friend constexpr bool operator==(
    const lazy_counted_iterator& x, default_sentinel_t);

template<common_with<I> I2>
    friend constexpr strong_ordering operator<==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)

friend constexpr iter_rvalue_reference_t<I> iter_move(const lazy_counted_iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current)));
template<indirectly_swappable<I> I2>
    friend constexpr void iter_swap(const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
        noexcept(noexcept(ranges::iter_swap(x.current, y.current)));

private:
    I current = I(); // exposition only
    iter_difference_t<I> length = 0; // exposition only
};
}

```

The member *typedef-name* `iterator_concept` denotes `forward_iterator_tag` if `I` models `forward_iterator`, and `input_iterator_tag` otherwise.

The member *typedef-name* `iterator_category` is defined if and only if `I` models `forward_iterator`. In that case, `lazy_counted_iterator<I>::iterator_category` denotes the type `iterator_traits<I>::iterator_category`.

§ 5.3.2. 24.5.2 Constructors and conversions [lazy.counted.iter.const]

```
constexpr lazy_counted_iterator(I i, iter_difference_t<I> n);
```

1. *Hardened preconditions*: `n >= 0` is true.

2. *Effects*: Initializes `current` with `std::move(i)` and `length` with `n`.

```
template<class I2>
    requires convertible_to<const I2&, I>
    constexpr lazy_counted_iterator(const lazy_counted_iterator<I2>& x);
```

3. *Effects*: Initializes `current` with `x.current` and `length` with `x.length`.

```
template<class I2>
    requires assignable_from<I&, const I2&>
    constexpr lazy_counted_iterator& operator=(const lazy_counted_iterator&
```

4. *Effects*: Assigns `x.current` to `current` and `x.length` to `length`.

5. *Returns*: `*this`.

§ 5.3.3. 24.5.4.3 Accessors [lazy.counted.iter.access]

```
constexpr iter_difference_t<I> count() const noexcept;
```

1. *Effects*: Equivalent to: `return length;`

§ 5.3.4. 24.5.4.4 Element access [lazy.counted.iter.elem]

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
    requires dereferenceable<const I>;
```

1. *Hardened preconditions*: `length > 0` is `true`.

2. *Effects*: Equivalent to: `return *current;`

§ 5.3.5. 24.5.4.5 Navigation [lazy.counted.iter.nav]

```
constexpr lazy_counted_iterator& operator++();
```

4. *Hardened preconditions*: `length > 0` is true.

5. *Effects*: Equivalent to:

```
if (length > 1) ++current;
--length;
return *this;
```

```
constexpr void operator++(int);
```

6. *Hardened preconditions*: `length > 0` is true.

7. *Effects*: Equivalent to `++*this`.

```
constexpr lazy_counted_iterator operator++(int)
    requires forward_iterator<I>;
```

8. *Effects*: Equivalent to:

```
lazy_counted_iterator tmp = *this;
++*this;
return tmp;
```

```
template<common_with<I> I2>
    friend constexpr iter_difference_t<I2> operator-(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

9. *Preconditions*: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

10. *Effects*: Equivalent to: `return y.length - x.length;`

```
friend constexpr iter_difference_t<I> operator-(
    const lazy_counted_iterator& x, default_sentinel_t);
```

11. *Effects*: Equivalent to: `return -x.length;`

```
friend constexpr iter_difference_t<I> operator-(
    default_sentinel_t, const lazy_counted_iterator& y);
```

12. *Effects*: Equivalent to: `return y.length;`

§ 5.3.6. 24.5.6 Comparisons [lazy.counted.iter.cmp]

```
template<common_with<I> I2>
    friend constexpr bool operator==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

1. *Preconditions*: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

2. *Effects*: Equivalent to: `return x.length == y.length;`

```
friend constexpr bool operator==(
    const lazy_counted_iterator& x, default_sentinel_t);
```

3. *Effects*: Equivalent to: `return x.length == 0;`

```
template<common_with<I> I2>
    friend constexpr strong_ordering operator<==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

4. *Preconditions*: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

5. *Effects*: Equivalent to: `return y.length <== x.length;`

[*Note 1*: The argument order in the *Effects*: element is reversed because `length` counts down, not up.
— end note]

§ 5.3.7. 24.5.7 Customizations [lazy.counted.iter.cust]

```
friend constexpr iter_rvalue_reference_t<I>
    iter_move(const lazy_counted_iterator& i)
        noexcept(noexcept(ranges::iter_move(i.current)));
```

1. *Hardened preconditions*: `i.length > 0` is true.

2. *Effects*: Equivalent to: `return ranges::iter_move(i.current);`

```
template<indirectly_swappable<I> I2>
    friend constexpr void
        iter_swap(const lazy_counted_iterator& x, const lazy_counted_iterator<
            noexcept(noexcept(ranges::iter_swap(x.current, y.current))));
```

3. *Hardened preconditions*: Both `x.length > 0` and `y.length > 0` are true.

4. *Effects*: Equivalent to `ranges::iter_swap(x.current, y.current)`.

§ 5.4.25.2 Header `<ranges>` synopsis [ranges.syn]

Modify the synopsis as follows:

```
// [...]
namespace std::ranges {
    // [...]
    // [range.take], take view
    template<view> class take_view;

    template<class T>
        constexpr bool enable_borrowed_range<take_view<T>> =
            enable_borrowed_range<T>;

    namespace views { inline constexpr unspecified take = unspecified; }

    // [range.lazy.take], lazy take view
    template<view> class lazy_take_view;

    template<class T>
        constexpr bool enable_borrowed_range<lazy_take_view<T>> =
            enable_borrowed_range<T>;

    namespace views { inline constexpr unspecified lazy_take = unspecified; }

    // [...]
    // [range.counted], counted view
    namespace views { inline constexpr unspecified counted = unspecified; }
```

```

// [range.lazy.counted], lazy counted view
namespace views { inline constexpr unspecified lazy_counted = unspecified; }

// [...]
// [range.as.input], as input view
template<input_range V>
    requires view<V>
class as_input_view;

template<class V>
    constexpr bool enable_borrowed_range<as_input_view<V>> =
        enable_borrowed_range<V>;

namespace views { inline constexpr unspecified as_input = unspecified; }

// [range.as.closed], as closed view
template<input_iterator I, sentinel_for<I> S>
class as_closed_view;

template<class I, class S>
    constexpr bool enable_borrowed_range<as_closed_view<I, S>> = true;

namespace views {
    inline constexpr unspecified as_closed = unspecified;
    inline constexpr unspecified closed_iota = unspecified;
}

// [...]
}

```

NOTE: Editor's Note: Add the following subclause to 25.7 Range adaptors [range.adaptors], after 25.7.10 Take view [range.take]

§ 5.5. 25.7. Lazy take view [range.lazy.take]

§ 5.5.1. 25.7. .1 Overview [range.lazy.take.overview]

1. `lazy_take_view` produces a view of the first N elements from another view, or all the elements if the adapted view contains fewer than N .

2. The name `views::lazy_take` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::lazy_take(E, F)` is ill-formed. Otherwise, the expression `views::lazy_take(E, F)` is expression-equivalent to:
- If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void)F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.
 - Otherwise, if `T` is a specialization of `optional` and `T` models `view`, then `(static_cast<D>(F) == D() ? ((void)E, T()) : decay-copy(E))`.
 - Otherwise, if `T` models `random_access_range` and `sized_range` and is a specialization of `span` (`[views.span]`), `basic_string_view` (`[string.view]`), or `ranges::subrange` (`[range.-subrange]`), then `U(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distance(E), F))`, except that `E` is evaluated only once, where `U` is a type determined as follows:
 - if `T` is a specialization of `span`, then `U` is `span<typename T::element_type>`;
 - otherwise, if `T` is a specialization of `basic_string_view`, then `U` is `T`;
 - otherwise, `T` is a specialization of `ranges::subrange`, and `U` is `ranges::subrange<iterator_t<T>>`;
 - otherwise, if `T` is a specialization of `iota_view` (`[range.iota.view]`) that models `random_access_range` and `sized_range`, then `iota_view(*ranges::begin(E), *(ranges::begin(E) + std::min<D>(ranges::distance(E), F)))`, except that `E` is evaluated only once.
 - Otherwise, if `T` is a specialization of `repeat_view` (`[range.repeat.view]`):
 - if `T` models `sized_range`, then `views::repeat(*E.value_, std::min<D>(ranges::distance(E), F))`, except that `E` is evaluated only once;
 - otherwise, `views::repeat(*E.value_, static_cast<D>(F))`.
 - Otherwise, `lazy_take_view(E, F)`.

[Example 1:

```
vector<int> is{0,1,2,3,4,5,6,7,8,9};
for (int i : is | views::lazy_take(5))
    cout << i << ' '; // prints 0 1 2 3 4
```

— end example]

§ 5.5.2.25.7.2 Class template `lazy_take_view` [range.lazy.take.view]

```
namespace std::ranges {
    template<view V>
    class lazy_take_view : public view_interface<lazy_take_view<V>> {
    private:
        V base_ = V(); // exposition only
        range_difference_t<V> count_ = 0; // exposition only

        // [range.lazy.take.sentinel], class template lazy_take_view::sentinel
        template<bool> class sentinel; // exposition only

    public:
        lazy_take_view() requires default_initializable<V> = default;
        constexpr lazy_take_view(V base, range_difference_t<V> count);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple_view<V>) {
            if constexpr (sized_range<V>) {
                if constexpr (random_access_range<V>) {
                    return ranges::begin(base_);
                } else {
                    auto sz = range_difference_t<V>(size());
                    return lazy_counted_iterator(ranges::begin(base_), sz);
                }
            } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>) {
                auto it = ranges::begin(base_);
                auto sz = std::min(count_, ranges::end(base_) - it);
                return lazy_counted_iterator(std::move(it), sz);
            } else {
                return lazy_counted_iterator(ranges::begin(base_), count_);
            }
        }

        constexpr auto begin() const requires range<const V> {
            if constexpr (sized_range<const V>) {
                if constexpr (random_access_range<const V>) {
                    return ranges::begin(base_);
                } else {
                    auto sz = range_difference_t<const V>(size());
                    return lazy_counted_iterator(ranges::begin(base_), sz);
                }
            }
        }
    };
}
```

```

    }
} else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<V>>) {
    auto it = ranges::begin(base_);
    auto sz = std::min(count_, ranges::end(base_) - it);
    return lazy_counted_iterator(std::move(it), sz);
} else {
    return lazy_counted_iterator(ranges::begin(base_), count_);
}
}

```

```

constexpr auto end() requires (!simple_view<V>) {
    if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
            return ranges::begin(base_) + range_difference_t<V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>)
        return default_sentinel;
    } else {
        return sentinel<false>{ranges::end(base_)};
    }
}

```

```

constexpr auto end() const requires range<const V> {
    if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>)
            return ranges::begin(base_) + range_difference_t<const V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>)
        return default_sentinel;
    } else {
        return sentinel<true>{ranges::end(base_)};
    }
}

```

```

constexpr auto size() requires sized_range<V> {
    auto n = ranges::size(base_);
    return ranges::min(n, static_cast<decltype(n)>(count_));
}

```

```

constexpr auto size() const requires sized_range<const V> {
    auto n = ranges::size(base_);

```

```

        return ranges::min(n, static_cast<decltype(n)>(count_));
    }

    constexpr auto reserve_hint() {
        if constexpr (approximately_sized_range<V>) {
            auto n = static_cast<range_difference_t<V>>(ranges::reserve_hint(base));
            return to_unsigned_like(ranges::min(n, count_));
        }
        return to_unsigned_like(count_);
    }

    constexpr auto reserve_hint() const {
        if constexpr (approximately_sized_range<const V>) {
            auto n = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base));
            return to_unsigned_like(ranges::min(n, count_));
        }
        return to_unsigned_like(count_);
    }
};

template<class R>
lazy_take_view(R&&, range_difference_t<R>)
    -> lazy_take_view<views::all_t<R>>;
}

```

```
constexpr lazy_take_view(V base, range_difference_t<V> count);
```

1. *Preconditions*: `count >= 0` is true.
2. *Effects*: Initializes `base_` with `std::move(base)` and `count_` with `count`.

§ 5.5.3. 25.7.3 Class template `take_view::sentinel` [range.lazy.take.sentinel]

```

namespace std::ranges {
    template<view V>
    template<bool Const>
    class lazy_take_view<V>::sentinel {
    private:
        using Base = maybe_const<Const, V>; // exposition only
        template<bool OtherConst>
            using CI = lazy_counted_iterator<iterator_t<maybe_const<OtherConst, V>>, V>;
    };
}

```

```

    sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only

public:
    sentinel() = default;
    constexpr explicit sentinel(sentinel_t<Base> end);
    constexpr sentinel(sentinel<!Const> s)
        requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

    constexpr sentinel_t<Base> base() const;

    friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

    template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst>>>
    friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);
};
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1. *Effects*: Initializes `end_` with `end`.

```

constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

2. *Effects*: Initializes `end_` with `std::move(s.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

3. *Effects*: Equivalent to: `return end_;`

```

friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

template<bool OtherConst = !Const>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst>>>
friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);

```

4. *Effects*: Equivalent to: `return y.count() == 0 || y.current == x.end_;`

NOTE: Editor's Note: Add the following subclause to 25.7 Range adaptors [range.adaptors], after 25.7.19 Counted view [range.counted]

§ 5.6. 25.7. Lazy counted view [range.lazy.counted]

1. A lazy counted view presents a view of the elements of the counted range ([iterator.requirements.general]) `i + [0, n)` for an iterator `i` and non-negative integer `n`.
2. The name `views::lazy_counted` denotes a customization point object ([customization-point.object]). Let `E` and `F` be expressions, let `T` be `decay_t<decltype((E))>`, and let `D` be `iter_difference_t<T>`. If `decltype((F))` does not model `convertible_to<D>`, `views::lazy_counted(E, F)` is ill-formed.

[Note 1: This case can result in substitution failure when `views::lazy_counted(E, F)` appears in the immediate context of a template instantiation. — end note]

Otherwise, `views::lazy_counted(E, F)` is expression-equivalent to:

- If `T` models `contiguous_iterator`, then `span(to_address(E), static_cast<size_t>(static_cast<D>(F)))`.
- Otherwise, if `T` models `random_access_iterator`, then `subrange(E, E + static_cast<D>(F))`, except that `E` is evaluated only once.
- Otherwise, `subrange(lazy_counted_iterator(E, F), default_sentinel)`.

NOTE: Editor's Note: Add the following subclause to 25.7 Range adaptors [range.adaptors], after 25.7.35 As input view [range.as.input]

§ 5.7. 25.7. As closed view [range.as.closed]

1. `as_closed_view` presents a view of an underlying sequence as if it represents a fully closed range.

[Note 1: A fully closed range is a range whose iterator is still valid to dereference even when comparing equal to its ending sentinel. — end note]

[Example 1:

```
vector<int> is{0,1,2,3,4,5,6,7,8,9};
for (int i : views::as_closed(is.begin() + 2, is.begin() + 5))
    cout << i << ' '; // prints 2 3 4 5
```

— end example]

2. The name `views::as_closed` denotes a range adaptor closure object ([range.adaptor.object]). Given subexpressions `E` and `F`, the expressions `views::as_closed(E)` and `views::as_closed(E, F)` are expression-equivalent to `as_closed_view(E)` and `as_closed_view(E, F)`, respectively.
3. The name `views::closed_iota` denotes a customization point object ([customization.point.object]). Given subexpressions `E` and `F`, the expressions `views::closed_iota(E, F)` is expression-equivalent to `views::as_closed(views::iota(E, F))`.

[Example 2:

```
for (int i : views::closed_iota(0, 5))
    cout << i << ' '; // prints 0 1 2 3 4 5
```

— end example]

§ 5.7.1. 25.7.1 Class template `as_closed_view` [range.as.closed.view]

```
namespace std::ranges {
    template <input_iterator I, sentinel_for<I> S>
    class as_closed_view : public view_interface<as_closed_view<I, S>> {
    private:
        // [range.as.closed.iterator], class iota_view::iterator
        class iterator; // exposition only

        I start_ = I(); // exposition only
        S end_   = S(); // exposition only

    public:
        as_closed_view() requires default_initializable<I> = default;
        template<range R>
            requires same_as<iterator_t<R>, I> && same_as<sentinel_t<R>, S>
            constexpr explicit as_closed_view(R&& r);
```

```
constexpr as_closed_view(I start, S end);

constexpr iterator begin() const;
constexpr auto end() const;
constexpr iterator end() const requires same_as<I, S>;

constexpr bool empty() const { return false; }
constexpr auto size() const requires sized_sentinel_for<S, I>;
};

template <class R>
as_closed_view(R&&)
    -> as_closed_view<iterator_t<R>, sentinel_t<R>>;
}
```

```
template<range R>
    requires same_as<iterator_t<R>, I> && same_as<sentinel_t<R>, S>
constexpr explicit as_closed_view(R&& r);
```

1. *Effects*: Initializes `start_` with `ranges::begin(r)` and `end_` with `ranges::end(r)`.

```
constexpr as_closed_view(I start, S end);
```

2. *Effects*: Initializes `start_` with `std::move(start)` and `end_` with `std::move(end)`.

```
constexpr iterator begin() const;
```

3. *Effects*: Equivalent to: `return iterator{start_, end_};`

```
constexpr auto end() const;
```

4. *Effects*: Equivalent to: `return default_sentinel;`

```
constexpr iterator end() const requires same_as<I, S>;
```

5. *Effects*: Equivalent to: `return iterator{end_, end_, true};`

```
constexpr auto size() const requires sized_sentinel_for<S, I>;
```

6. *Effects*: Equivalent to: `return end_ - start_ + 1;`

§ 5.7.2. 25.7.2 Class `as_closed_view::iterator` [range.as.closed.iterator]

```
namespace std::ranges {
    template <input_iterator I, sentinel_for<I> S>
    class as_closed_view<I, S>::iterator {
    private:
        I current_ = I();           // exposition only
        S last_ = S();              // exposition only
        bool is_end_ = false;       // exposition only

        constexpr iterator(I current, S last, bool is_end);

    public:
        using iterator_concept = see below;
        using iterator_category = see below; // not always present
        using value_type = iter_value_t<I>;
        using difference_type = iter_difference_t<I>;

        iterator() requires default_initializable<I> = default;
        constexpr iterator(I current, S last);

        constexpr decltype(auto) operator*() const { return *current_; }
        constexpr decltype(auto) operator*() { return *current_; }
        constexpr iterator& operator++();
        constexpr void operator++(int);
        constexpr iterator operator++(int)
            requires forward_iterator<I>;
        constexpr iterator& operator--()
            requires bidirectional_iterator<I>;
        constexpr iterator operator--(int)
            requires bidirectional_iterator<I>;

        constexpr iterator& operator+=(difference_type n)
            requires random_access_iterator<I>;
```

```

constexpr iterator& operator--(difference_type n)
    requires random_access_iterator<I>;
constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_iterator<I>;
constexpr decltype(auto) operator[](difference_type n)
    requires random_access_iterator<I>;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<I>;
friend constexpr bool operator==(const iterator& x, default_sentinel_t)
    requires equality_comparable<I>;
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_iterator<I>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_iterator<I>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_iterator<I>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_iterator<I>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_iterator<I> && three_way_comparable<I>;

friend constexpr iterator operator+(iterator i, difference_type n)
    requires random_access_iterator<I>;
friend constexpr iterator operator+(difference_type n, iterator i)
    requires random_access_iterator<I>;
friend constexpr iterator operator-(iterator i, difference_type n)
    requires random_access_iterator<I>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<S, I>;
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
    requires sized_sentinel_for<S, I>;
friend constexpr difference_type operator-(default_sentinel_t, const iterator& y)
    requires sized_sentinel_for<S, I>;
};
}

```

1. `iterator::iterator_concept` is defined as follows:

- If `I` models `random_access_iterator`, then `iterator_concept` denotes `random_access_iterator_tag`.
- Otherwise, if `I` models `bidirectional_iterator`, then `iterator_concept` denotes `bidirectional_iterator_tag`.

- Otherwise, if `I` models `forward_iterator`, then `iterator_concept` denotes `forward_iterator_tag`.
 - Otherwise, `iterator_concept` denotes `input_iterator_tag`.
2. The member typedef-name `iterator_category` is defined if and only if `I` models `forward_iterator`. In that case, `iterator::iterator_category` is defined as follows: Let `C` denote the type `iterator_traits<I>::iterator_category`.
- If `C` models `derived_from<contiguous_iterator_tag>`, `iterator_category` denotes `random_access_iterator_tag`;
 - Otherwise, `iterator_category` denotes `C`.

```
constexpr iterator(I current, S last, bool is_end);
```

3. *Effects*: Initializes `current_` with `std::move(current)`, `last_` with `std::move(last)`, and `is_end_` with `is_end`.

```
constexpr iterator(I current, S last);
```

4. *Effects*: Initializes `current_` with `std::move(current)` and `last_` with `std::move(last)`.

```
constexpr iterator& operator++();
```

5. *Effects*: Equivalent to:

```
if (current_ == last_) is_end_ = true;
else ++current_;
return *this;
```

```
constexpr void operator++(int);
```

6. *Effects*: Equivalent to `++*this;`.

```
constexpr iterator operator++(int)
    requires forward_iterator<I>;
```

7. *Effects*: Equivalent to:

```
auto tmp = *this;  
++*this;  
return tmp;
```

```
constexpr iterator& operator--()  
    requires bidirectional_iterator<I>;
```

8. *Effects*: Equivalent to:

```
if (is_end_) is_end_ = false;  
else --current_;  
return *this;
```

```
constexpr iterator operator--(int)  
    requires bidirectional_iterator<I>;
```

9. *Effects*: Equivalent to:

```
auto tmp = *this;  
--*this;  
return tmp;
```

```
constexpr iterator& operator+=(difference_type n)  
    requires random_access_iterator<I>;
```

10. *Effects*: Equivalent to:

```
if (n >= 1 && current_ + (n - 1) == last_) {  
    current_ += n - 1;  
    is_end_ = true;
```

```

}
else current_ += n;
return *this;

```

```

constexpr iterator& operator--(difference_type n)
    requires random_access_iterator<I>;

```

11. *Effects*: Equivalent to:

```

if (is_end_ && n >= 1) {
    is_end_ = false;
    --n;
}
current_ -= n;
return *this;

```

```

constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_iterator<I>;
constexpr decltype(auto) operator[](difference_type n)
    requires random_access_iterator<I>;

```

12. *Effects*: Equivalent to: `return current_[n];`

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<I>;

```

13. *Effects*: Equivalent to:

```

return x.current_ == y.current_ && x.last_ == y.last_ && x.is_end_ == y.is_end_;

```

```

friend constexpr bool operator==(const iterator& x, default_sentinel_t) noexcept;

```

14. *Effects*: Equivalent to: `return x.is_end_;`

```

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_iterator<I>;

```

15. *Effects*: Equivalent to:

```
return x.current_ < y.current_ || (!x.is_end_ && y.is_end_);
```

```
friend constexpr bool operator>(const iterator& x, const iterator& y)  
    requires random_access_iterator<I>;
```

16. *Effects*: Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)  
    requires random_access_iterator<I>;
```

17. *Effects*: Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)  
    requires random_access_iterator<I>;
```

18. *Effects*: Equivalent to: `return !(x < y);`

```
friend constexpr auto operator<==(const iterator& x, const iterator& y)  
    requires random_access_iterator<I> && three_way_comparable<I>;
```

19. *Effects*: Equivalent to:

```
if (x.is_end_ != y.is_end_) return x.is_end_ <=> y.is_end_;  
return x.current_ <=> y.current_;
```

```
friend constexpr iterator operator+(iterator i, difference_type n)  
    requires random_access_iterator<I>;  
friend constexpr iterator operator+(difference_type n, iterator i)  
    requires random_access_iterator<I>;
```

20. *Effects*: Equivalent to: `return i += n;`

```
friend constexpr iterator operator-(iterator i, difference_type n)  
    requires random_access_iterator<I>;
```

21. *Effects*: Equivalent to: `return i -= n;`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires sized_sentinel_for<S, I>;
```

22. *Effects*: Equivalent to: `return x.current_ + x.is_end_ - y.current_ - y.is_end_;`

```
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
requires sized_sentinel_for<S, I>;
```

23. *Effects*: Equivalent to: `return x.current_ + x.is_end_ - x.last_;`

```
friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
requires sized_sentinel_for<S, I>;
```

24. *Effects*: Equivalent to: `return x.last_ - x.current_ - x.is_end_;`

§ References

§ Normative References

[N5032]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025.
URL: <https://wg21.link/n5032>

[P2406R5]

Yehezkel Bernat, Yehuda Bernat. *Add lazy counted iterator*. 8 February 2023. URL:
<https://wg21.link/p2406r5>

[P2799R0]

Tim Song. *Closed ranges may be a problem; breaking counted_iterator is not the solution*. 14
February 2023. URL: <https://wg21.link/p2799r0>

§ Informative References

[D2578R0]

Yehezkel Bernat, Yehuda Bernat. *Block eager input (non-forward) iterators from counted_iterator*. 18 April 2022. URL: <https://isocpp.org/files/papers/D2578R0.html>

[P1642R11]

Ben Craig. *Freestanding Library: Easy [utilities], [ranges], and [iterators]*. 1 July 2022. URL: <https://wg21.link/p1642r11>

[P2210R2]

Barry Revzin. *Superior String Splitting*. 5 March 2021. URL: <https://wg21.link/p2210r2>

[P3060R1]

Weile Wei, Zhihao Yuan. *Add std::views::upto(n)*. 15 February 2024. URL: <https://wg21.link/p3060r1>

[P3828R1]

Nicolai Josuttis. *Rename the to_input view to as_input*. 7 March 2026. URL: <https://wg21.link/p3828r1>

[US46-107]

US National Body. *[counted.iterator] Too many iterator increments*. 31 October 2022. URL: <https://github.com/cplusplus/nballot/issues/523>